Internal Implementations of Hash Maps, Collision Resolution Strategies, Dynamic Resizing, and String Grouping Algorithms: A Formal Algorithmic Analysis

Abstract

This paper presents a formal, unified algorithmic analysis of hash map internal implementations derived from classroom instruction on May 20, 2026. We trace the theoretical progression from elementary linear array searching to Direct Access Tables (DAT), identify the memory footprint limitations of DATs, and formalize hash functions as a dynamic compression mechanism. Using the Pigeonhole Principle, we prove the mathematical inevitability of hash collisions. We analyze collision resolution strategies—comparing separate chaining with open addressing techniques—and evaluate modern optimizations including Java 8+ bucket treeification into Red-Black trees. Furthermore, we formalize load factor thresholding (lambda) and prove through aggregate amortized analysis that dynamic table doubling achieves O(1) amortized insertion complexity. A complete, compilable reference implementation of a generic HashMap in Java is provided. Finally, we apply hash mapping to the Group Anagrams problem, contrasting O(N K log K) sorting keys with optimal O(N K) frequency-tuple encodings.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Hash-based data structures represent one of the foundational paradigms of computer science, bridging theoretical efficiency and practical systems engineering. By establishing an associative mapping from an arbitrary key domain $\mathcal{K}$ to a value domain $\mathcal{V}$, hash maps enable constant-time expected data access, search, and insertion operations. Unlike comparison-based data structures such as balanced binary search trees, which impose a lower bound of $\Omega(\log N)$ comparison operations for basic dictionary primitives, hash maps exploit direct array indexing to achieve $O(1)$ expected time complexity.

This paper provides a formal, unified algorithmic analysis of the internal mechanics of hash maps, derived from classroom instruction on May 20, 2026. We systematically trace the evolution of search paradigms from elementary linear arrays to Direct Access Tables (DAT), identify the structural bottlenecks of DATs, and introduce hash functions as a dynamic compression mechanism. We analyze collision resolution strategies, prove the necessity of collisions via the Pigeonhole Principle, evaluate load factor thresholding (λ), and derive the $O(1)$ amortized time complexity of dynamic table resizing (rehashing).

Furthermore, we present a complete, reference implementation of a generic HashMap data structure with separate chaining, dynamic rehashing, and load factor evaluation. Finally, we apply these concepts to the classic Group Anagrams problem, comparing $O(N \cdot K \log K)$ sorting-key techniques with optimal $O(N \cdot K)$ frequency-tuple encoding strategies.

1.1 Preliminaries and Notation

Throughout this paper, we adopt the following mathematical and algorithmic notation:

- N: The number of elements inserted into the dictionary structure.
- M: The capacity of the underlying hash table array (bucket count).
- $\mathcal{K}$: The domain of all possible keys.
- $\mathcal{V}$: The domain of stored value payloads.
- $h : \mathcal{K} \to \{0, 1, \ldots, M-1\}$: The primary hash function mapping key $k \in \mathcal{K}$ to a bucket index.
- $\lambda = \frac{N}{M}$: The load factor of the hash table.

2 Search Paradigms: Linear Arrays vs. Direct Access Tables (DAT)

To understand the motivation behind hash maps, we first examine the fundamental problem of membership verification across discrete datasets.

2.1 Formal Problem Statement

Given a static data array $A = [a_0, a_1, \ldots, a_{N-1}]$ of size N and a query array $Q = [q_0, q_1, \ldots, q_{M-1}]$ of size M, construct an algorithm to verify whether each query element $q_j \in Q$ exists in A, returning a boolean result vector $R \in \{\text{True}, \text{False}\}^M$.

2.2 Approach I: Brute-Force Linear Scan

The naive approach performs an unindexed linear traversal across array A for every query $q_j \in Q$.

```
boolean[] searchLinear(int[] A, int[] Q) {
    boolean[] res = new boolean[Q.length];
    for (int j = 0; j < Q.length; j++) {
        boolean found = false;
        for (int i = 0; i < A.length; i++) {
            if (A[i] == Q[j]) {
                found = true;
                break;
            }
        }
        res[j] = found;
    }
    return res;
}
```

Theorem 1 (Complexity of Linear Search). The brute-force search algorithm requires $O(N \cdot M)$ worst-case time complexity and $O(1)$ auxiliary space complexity.

Proof. For each of the M query elements, the inner loop traverses up to N elements of array A in the event of a missing key or tail placement. Summing over all queries yields $T(N, M) = \sum_{j=1}^{M} O(N) = O(N \cdot M)$. Auxiliary space is restricted to temporary loop variables, yielding $O(1)$. □

2.3 Approach II: Sorting and Binary Search

When array A can be preprocessed, we can sort A in non-decreasing order and perform binary search for each query $q_j \in Q$.

```
boolean[] searchBinary(int[] A, int[] Q) {
    Arrays.sort(A); // O(N log N)
    boolean[] res = new boolean[Q.length];
    for (int j = 0; j < Q.length; j++) {
        res[j] = binarySearch(A, Q[j]); // O(log N)
    }
    return res;
}
```

Theorem 2 (Complexity of Binary Search Lookup). The sorting and binary search paradigm achieves $O(N \log N + M \log N)$ total time complexity and $O(1)$ auxiliary space complexity (assuming in-place sorting like HeapSort or QuickSort).

2.4 Approach III: Direct Access Table (DAT)

When keys are strictly non-negative integers bounded by a known maximum value $K_{\max} = \max_i A[i]$, we can construct a boolean direct access table DAT of size $K_{\max} + 1$, where element index directly encodes key presence.

```
boolean[] searchDAT(int[] A, int[] Q) {
    int maxEle = 0;
    for (int x : A) if (x > maxEle) maxEle = x;

    boolean[] dat = new boolean[maxEle + 1];
    for (int x : A) {
        dat[x] = true; // O(1) direct write
    }

    boolean[] res = new boolean[Q.length];
    for (int j = 0; j < Q.length; j++) {
        if (Q[j] >= 0 && Q[j] <= maxEle) {
            res[j] = dat[Q[j]]; // O(1) direct read
        } else {
            res[j] = false;
        }
    }
    return res;
}
```

Theorem 3 (Complexity of Direct Access Table). The Direct Access Table achieves $O(N + M)$ total time complexity and $O(K_{\max})$ space complexity.

Proof. Populating the table takes $O(N)$ time across array A. Answering M queries takes $O(1)$ time per query, totaling $O(M)$ operations. Overall time is $O(N + M)$. Space is proportional to the maximum numerical key value $K_{\max} + 1$. □

2.5 Structural Bottlenecks of Direct Access Tables

While DAT achieves optimal $O(1)$ lookup per query, it suffers from severe real-world constraints:

1) Extreme Space Inefficiency for Sparse Domains: If $A = [1, 10^9]$, the DAT requires an array of size $10^9 + 1$ (≈ 1 GB of memory) to store just two elements, yielding a space utilization efficiency of $\eta = \frac{N}{K_{\max}+1} \approx 2 \times 10^{-9}$.
2) Domain Restriction: DAT fails natively for negative integers (requires offset shifts), floating-point numbers, and composite data types such as strings or custom objects.
3) Memory Allocation Failures: Large key bounds easily trigger memory exhaustion errors (OutOfMemoryError).

3 Hash Functions, Compression, and Collision Mechanics

To maintain the $O(1)$ address-calculation speed of DAT while overcoming its memory footprint limitations, hash maps introduce a two-stage transformation: key hashing followed by range compression.

3.1 Compression via Modulo Arithmetic

A hash function $h(k)$ maps an arbitrary key $k \in \mathcal{K}$ to a bounded array bucket index in $[0, M - 1]$. The standard modulo compression function is defined as:

$$h(k) = |g(k)| \pmod{M}$$

where $g(k)$ is an integer hash code generator (e.g., hashCode() in Java).

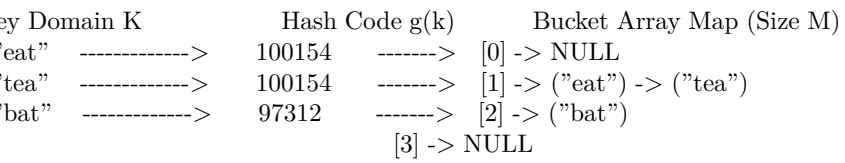


Fig. 1. Conceptual transformation of key string inputs into compressed bucket array indices through hash codes and modulo arithmetic.

3.2 The Pigeonhole Principle and Mathematical Proof of Collisions

Definition 4 (Hash Collision). A collision occurs when two distinct keys $k_1, k_2 \in \mathcal{K}$ ($k_1 \neq k_2$) evaluate to the exact same table bucket index under h, such that $h(k_1) = h(k_2)$.

Theorem 5 (Inevitability of Hash Collisions). For any hash function $h : \mathcal{K} \to \{0, 1, \ldots, M - 1\}$, if the size of the key universe $|\mathcal{K}| > M$, hash collisions are mathematically guaranteed to exist.

Proof. By the Dirichlet Pigeonhole Principle, if N items (pigeons) are placed into M containers (pigeonholes) where $N > M$, at least one container must hold more than one item. Letting $\mathcal{K}$ represent the set of pigeons and $\{0, \ldots, M - 1\}$ represent the set of pigeonholes, mapping $|\mathcal{K}|$ elements into M indices guarantees that $\exists k_1, k_2 \in \mathcal{K}, k_1 \neq k_2 \implies h(k_1) = h(k_2)$. □

4 Collision Resolution Strategies

Since collisions cannot be avoided when compressing large key spaces, hash maps employ formal collision resolution strategies. These are classified into two broad categories: Open Hashing (Separate Chaining) and Closed Hashing (Open Addressing).

4.1 Separate Chaining (Open Hashing)

In separate chaining, each index of the hash table array M maintains a head pointer to a dynamic linked list (or dynamic vector) containing all key-value entries hashing to that bucket.

4.1.1 Algorithmic Operations under Chaining

- Insertion (put): Compute $bi = h(k)$. Traverse chain at map[bi]. If key k exists, overwrite value. Otherwise, append new node (k, v) to chain.
- Search (get): Compute $bi = h(k)$. Traverse chain at map[bi] linearly comparing key equality. Return value if found, else null / -1.
- Deletion (delete): Compute $bi = h(k)$. Search chain for node with key k, unlink node from sequence, and decrement total size.

4.1.2 Worst-Case Degradation and Java 8+ Treeification Optimization

In the worst-case scenario where a poor hash function maps all N keys to the same bucket index $bi = 0$, separate chaining degrades to a single linear linked list of length N, causing lookup operations to deteriorate to $O(N)$ time.

To mitigate adversarial $O(N)$ attacks, modern production engines (such as Java 8+ java.util.HashMap) implement dynamic treeification:

- When the node count of a single bucket chain exceeds TREEIFY_THRESHOLD (default $\gamma = 8$), the singly-linked list is converted into a Red-Black Tree (Self-Balancing BST).
- If table resizing subsequently reduces chain length below UNTREEIFY_THRESHOLD (default $\delta = 6$), the tree is converted back to a linked list.

Theorem 6 (Treeified Chaining Complexity). Under Red-Black Tree bucket treeification, the worst-case search, insertion, and deletion time complexity is bounded by $O(\log N)$.

4.2 Open Addressing (Closed Hashing)

In open addressing, all elements are stored directly within the single primary hash table array without external dynamic chains. When a collision occurs at index $h(k)$, probe sequences search alternative slots according to a probing function $p(k, i)$:

$$h(k, i) = (h'(k) + p(k, i)) \pmod{M}, \quad \text{for probe step } i = 0, 1, 2, \ldots$$

1) Linear Probing: $p(k, i) = i$. Probes sequential buckets $h'(k), h'(k)+1, h'(k)+2, \ldots$. Suffers from Primary Clustering, where long contiguous blocks of occupied slots build up, drastically increasing probe lengths.
2) Quadratic Probing: $p(k, i) = c_1 i + c_2 i^2$. Reduces primary clustering by inspecting slots at quadratic offsets. Suffers from Secondary Clustering when two keys share the same initial hash code $h'(k)$.
3) Double Hashing: $p(k, i) = i \cdot h_2(k)$, where $h_2(k)$ is an independent secondary hash function ($h_2(k) \neq 0$). Provides optimal uniform probing distribution by varying step size based on the key itself.

5 Load Factor (λ) and Amortized Analysis of Dynamic Rehashing

The efficiency of collision resolution techniques depends critically on table occupancy.

5.1 Mathematical Definition of Load Factor

Definition 7 (Load Factor). The load factor λ of a hash table with N elements and bucket capacity M is defined as:

$$\lambda = \frac{N}{M}$$

As λ increases, the probability of collisions rises exponentially, leading to longer search paths.

5.2 Dynamic Resizing (Rehashing) Algorithm

To maintain $O(1)$ expected performance, the hash map monitors λ. When λ exceeds a predefined maximum threshold $\lambda_{\max}$ (typically 0.75 or 1.0), the map automatically triggers a rehash() operation:

1) Allocate a new bucket array of capacity $M_{\text{new}} = 2 \times M_{\text{old}}$.
2) Re-evaluate the bucket index $h_{\text{new}}(k) = |g(k)| \pmod{M_{\text{new}}}$ for every existing node across all old chains.
3) Re-insert nodes into their respective new buckets.
4) Replace the old bucket array reference with the new array and set $M \leftarrow M_{\text{new}}$.

5.3 Amortized Time Complexity Proof via the Aggregate Method

Theorem 8 (Amortized $O(1)$ Insertion Complexity). A sequence of N put operations on an initially empty hash map with dynamic doubling capacity requires $O(N)$ total execution time, yielding an amortized time complexity of $O(1)$ per operation.

Proof. Assume table capacity doubles whenever N reaches capacity M. Let the initial capacity be $M_0 = 1$. The cost c_i of the i-th insertion operation is:

$$c_i = \begin{cases} i & \text{if } i-1 \text{ is a power of 2 (triggers resize of size } i-1 \text{ plus insertion)} \\ 1 & \text{otherwise} \end{cases}$$

The total cost $T(N)$ for N insertions is the sum of standard insertion costs plus all resizing costs:

$$T(N) = \sum_{i=1}^{N} c_i = N + \sum_{j=0}^{\lfloor \log_2 N \rfloor} 2^j$$

Using the geometric series formula $\sum_{j=0}^{k} 2^j = 2^{k+1} - 1$:

$$T(N) \leq N + 2^{\lfloor \log_2 N \rfloor + 1} - 1 < N + 2N = 3N = O(N)$$

Dividing the total work $T(N)$ by N operations yields the amortized cost per operation:

$$T_{\text{amortized}} = \frac{T(N)}{N} \leq \frac{3N}{N} = 3 = O(1)$$

Thus, dynamic rehashing achieves $O(1)$ amortized insertion time. □

6 Complete Implementation of Generic HashMap in Java

Below is a complete, reference implementation of a custom HashMap class implementing separate chaining, dynamic rehashing, load factor checking, and full dictionary primitives.

```
import java.util.ArrayList;
import java.util.List;

public class CustomHashMap<K, V> {

    // Internal node structure for separate chaining
    private class HNode {
        K key;
        V value;

        public HNode(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private List<HNode>[] map;
    private int totalSize;
    private int bucketCount;
    private static final double MAX_LOAD_FACTOR = 0.75;

    @SuppressWarnings("unchecked")
    public CustomHashMap(int initialCapacity) {
        this.bucketCount = initialCapacity;
        this.totalSize = 0;
        this.map = new List[bucketCount];
        for (int i = 0; i < bucketCount; i++) {
            this.map[i] = new ArrayList<>();
        }
    }

    public CustomHashMap() {
        this(10); // Default initial capacity of 10
    }

    // Compression function: maps key hash code to bucket index
    private int getBucketIndex(K key) {
        int hashCode = (key == null) ? 0 : key.hashCode();
        return Math.abs(hashCode) % bucketCount;
    }

    // Linearly searches bucket list for node matching key
    private int getItemIndex(int bi, K key) {
        List<HNode> bucket = map[bi];
        for (int i = 0; i < bucket.size(); i++) {
            K currentKey = bucket.get(i).key;
            if ((key == null && currentKey == null) ||
                (key != null && key.equals(currentKey))) {
```

```
            return i;
        }
    }
    return -1; // Key not found
}

public double getLoadFactor() {
    return (double) totalSize / bucketCount;
}

// Inserts or updates key-value pair
public void put(K key, V value) {
    int bi = getBucketIndex(key);
    int itemIndex = getItemIndex(bi, key);

    if (itemIndex != -1) {
        // Case 1: Key already exists -> Update value
        map[bi].get(itemIndex).value = value;
    } else {
        // Case 2: Key does not exist -> Insert new node
        if (getLoadFactor() >= MAX_LOAD_FACTOR) {
            rehash();
            bi = getBucketIndex(key); // Re-evaluate bucket index after resizing
        }
        map[bi].add(new HNode(key, value));
        totalSize++;
    }
}

// Retrieves value associated with key
public V get(K key) {
    int bi = getBucketIndex(key);
    int itemIndex = getItemIndex(bi, key);

    if (itemIndex == -1) {
        return null; // Key does not exist
    }
    return map[bi].get(itemIndex).value;
}

// Removes key-value pair
public V remove(K key) {
    int bi = getBucketIndex(key);
    int itemIndex = getItemIndex(bi, key);

    if (itemIndex == -1) {
        return null;
    }
    HNode removedNode = map[bi].remove(itemIndex);
    totalSize--;
    return removedNode.value;
}

public boolean containsKey(K key) {
    int bi = getBucketIndex(key);
    return getItemIndex(bi, key) != -1;
}

public int size() {
    return totalSize;
}
```

```
    // Doubles table capacity and re-inserts all existing entries
    @SuppressWarnings("unchecked")
    private void rehash() {
        List<HNode>[] oldMap = map;
        int oldBucketCount = bucketCount;

        bucketCount = oldBucketCount * 2;
        map = new List[bucketCount];
        for (int i = 0; i < bucketCount; i++) {
            map[i] = new ArrayList<>();
        }
        totalSize = 0; // Reset size, as put() will increment totalSize

        for (int i = 0; i < oldBucketCount; i++) {
            List<HNode> bucket = oldMap[i];
            for (HNode node : bucket) {
                put(node.key, node.value); // Re-insert into enlarged table
            }
        }
    }
}
```

7 Algorithmic Application: Group Anagrams

Hash maps serve as essential building blocks for solving complex algorithmic tasks. A classic problem illustrating string hashing is Group Anagrams.

7.1 Problem Statement

Given an array of strings $S = [s_0, s_1, \ldots, s_{N-1}]$, group all anagrams together in any order. An anagram is a word formed by rearranging the letters of another word using all original letters exactly once.

Example Input: $S =$ ["eat", "tea", "tan", "ate", "nat", "bat"]
Expected Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

7.2 Method I: Canonical Character Sorting Key

Since anagrams contain identical character frequencies, sorting the characters of any word produces a unique canonical representation shared by all words in its anagram class (e.g., sort("eat") = sort("tea") = "aet").

```
public List<List<String>> groupAnagramsSorting(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] ca = s.toCharArray();
        Arrays.sort(ca); // O(K log K)
        String key = String.valueOf(ca);

        if (!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Theorem 9 (Complexity of Sorting-Key Anagram Grouping). Sorting-key anagram grouping runs in $O(N \cdot K \log K)$ time and requires $O(N \cdot K)$ auxiliary space, where N is the number of strings and K is the maximum string length.

7.3 Method II: Character Frequency Tuple Key (Optimal Linear Time)

To eliminate the $O(K \log K)$ sorting penalty per string, we construct a 26-element character frequency array for lowercase English letters and format it into a string key (e.g., "#1#0#0...#1").

```
public List<List<String>> groupAnagramsFrequency(String[] strs) {
```

```
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }

        // Build unique string encoding key: e.g. "#1#0#0...#1"
        StringBuilder sb = new StringBuilder("");
        for (int i = 0; i < 26; i++) {
            sb.append('#');
            sb.append(count[i]);
        }
        String key = sb.toString();

        if (!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Theorem 10 (Complexity of Frequency-Tuple Anagram Grouping). Frequency-tuple anagram grouping runs in $O(N \cdot K)$ optimal time complexity and $O(N \cdot K)$ auxiliary space.

Proof. Counting character frequencies across string s_i takes $O(K)$ operations. Building the 26-element key takes $O(26) = O(1)$ time. Hash table lookup and insertion take $O(1)$ expected time. Iterating over N strings yields a total time bound of $\sum_{i=1}^{N} O(K) = O(N \cdot K)$. □

8 Comparative Complexity Analysis

Table 1 presents a comprehensive performance comparison of membership lookup and dictionary structures discussed in this paper.

TABLE 1
Comprehensive Asymptotic Complexity Comparison of Lookup Structures

Data Structure / Algorithm	Search (Avg)	Search (Worst)	Space Complexity	Domain Bounds
Unindexed Array Search	$O(N)$	$O(N)$	$O(1)$ Auxiliary	Unrestricted
Sorted Array + Binary Search	$O(\log N)$	$O(\log N)$	$O(1)$ Auxiliary	Orderable Domains
Direct Access Table (DAT)	$O(1)$	$O(1)$	$O(K_{\max})$	Bounded Non-negative Integers
HashMap (Separate Chaining - List)	$O(1)$	$O(N)$	$O(N + M)$	Universal Hashable Keys
HashMap (Separate Chaining - Treeified)	$O(1)$	$O(\log N)$	$O(N + M)$	Universal Comparable Keys
HashMap (Open Addressing - Linear Probe)	$O(1)$	$O(N)$	$O(M)$	Universal Hashable Keys
Dynamic HashMap (Rehashed Amortized)	$O(1)$	$O(1)$ Amortized	$O(N)$	Universal Hashable Keys

9 Real-World Systems Engineering Applications

The theoretical concepts analyzed in this paper directly underpin key modern soft infrastructure:

1) Database Indexing Engines: Hash indexes in PostgreSQL and MySQL (Memory engine) create $O(1)$ point-lookup indices for equality checks (WHERE id = x), bypassing tree traversal overhead.
2) High-Performance Caches: Distributed in-memory caching systems such as Redis and Memcached utilize dynamically resized hash tables to achieve sub-millisecond key-value retrieval.
3) Compiler Symbol Tables: Programming language compilers construct symbol tables via hash maps to bind variable identifiers to memory addresses and type signatures across lexer/parser stages.
4) Bioinformatics & Genomics: K-mer frequency analysis and sequence alignment algorithms employ hash maps to index DNA substring sequences across genome assemblies.

10 Conclusion

This paper has presented a rigorous, unified formal analysis of hash maps, collision resolution mechanisms, load factor thresholding, dynamic rehashing, and string grouping algorithms. We demonstrated that while Direct Access Tables provide constant-time indexing, their memory footprint makes them unusable for large or sparse domains. Hash maps resolve this by compressing keys via modulo arithmetic, utilizing the Pigeonhole Principle to prove collision inevitability, and employing separate chaining or open addressing to guarantee performance stability.

We established through aggregate amortized analysis that table doubling achieves $O(1)$ amortized insertion time. Finally, we demonstrated the algorithmic utility of hash maps in string processing, proving that frequency-tuple encoding achieves optimal $O(N \cdot K)$ time complexity for anagram grouping.